ALBUKHARY INTERNATIONAL UNIVERSITY

DU014 (K)

CRYPTOGRAPHY ESSENTIALS

SYSTEM DESIGN

AES-Based Secure File Encryption and Decryption Tool

Student Name	_____Maralbyek Tilyek_____
Student ID	_____AIU24102280_____
Course / Course Code	Cryptography Essentials CCC2243
Project Type	Individual Project
Year / Semester	Year 2, Semester 3, 2025-2026

Table of Content

1. Introduction

This system design report presents the detailed design of the proposed AES-Based Secure File Encryption and Decryption Tool. The system is a local desktop application intended to protect files using password-derived encryption keys and authenticated encryption. The design is based on the project proposal and literature review and translates the selected cryptographic concepts into an implementable application structure.

The proposed system uses Python as the programming language, Tkinter for the graphical user interface, PBKDF2-HMAC-SHA256 for password-based key derivation, and AES-256-GCM for authenticated encryption. The system is designed for a single user on a single computer and does not require cloud storage, network transmission, or a separate key-management server.

2. System Overview

The application accepts either a normal file for encryption or an encrypted .enc file for decryption. During encryption, the user selects a file and supplies a password. The system generates a random salt and nonce, derives a 256-bit AES key from the password, encrypts the file using AES-256-GCM, and stores the required non-secret parameters together with the encrypted data in a defined binary file format.

During decryption, the application reads the encrypted file, extracts the stored salt, nonce, authentication tag, and ciphertext, derives the same AES key from the supplied password, and performs AES-GCM authentication and decryption. If authentication succeeds, the original file is reconstructed. If authentication fails, the application does not release or save decrypted output.

3. System Requirements

3.1 Functional Requirements

ID	Requirement	Description
FR1	File Selection	The system shall allow the user to select a file through a file-picker dialog.
FR2	Password Input	The system shall accept a password and mask the password characters in the interface.
FR3	Password Confirmation	The encryption interface shall request password confirmation to reduce typing errors.
FR4	Encryption	The system shall derive an AES-256 key and encrypt the selected file using AES-GCM.
FR5	Encrypted File Creation	The system shall create a separate encrypted output file with a .enc extension.

FR6	Decryption	The system shall decrypt an encrypted file after successful authentication.
FR7	Integrity Verification	The system shall reject an incorrect password or modified encrypted data.
FR8	Output Protection	The system shall avoid creating final decrypted output until authentication succeeds.
FR9	Status Messages	The system shall display clear success and error messages.
FR10	Error Handling	The system shall handle missing files, invalid formats, permission errors, and interrupted operations without crashing.

3.2 Non-Functional Requirements

Category	Requirement
Security	Use AES-256-GCM, random salts, fresh nonces, and PBKDF2-HMAC-SHA256.
Usability	Provide a simple graphical interface understandable by a non-technical user.
Reliability	Successful encryption and decryption shall preserve file contents exactly.
Performance	The implementation shall be evaluated across multiple file sizes, including large files.
Maintainability	Cryptographic operations shall be separated from the GUI where practical.
Portability	The application shall run on supported Python 3.10+ environments with required dependencies.

4. System Architecture

The application follows a layered logical structure. The presentation layer contains the Tkinter interface and collects user input. The application layer controls the encryption and decryption workflow, validates input, manages file operations, and reports results. The cryptographic layer performs PBKDF2 key derivation and AES-256-GCM operations through the Python cryptography library. The file layer handles reading and writing the encrypted file format.

Layer	Main Responsibility
Presentation Layer	Tkinter GUI, file selection, password entry, progress/status messages.
Application Layer	Workflow control, validation, temporary-file handling, success/failure decisions.
Cryptographic and File Layer	PBKDF2, AES-GCM, secure random generation, encrypted file formatting and parsing.

5. System Design Diagrams

Three diagrams are used to describe the system from different perspectives. The Use Case Diagram shows what the user can do with the system. The Data Flow Diagram shows how information moves between processes and data storage. The Application Flow Diagram shows the sequence of operations and decisions performed during encryption and decryption.

5.1 Use Case Diagram

The Use Case Diagram identifies the user as the primary external actor. The main use cases are selecting a file, entering a password, encrypting a file, decrypting an encrypted file, selecting an output location, and receiving success or error feedback.

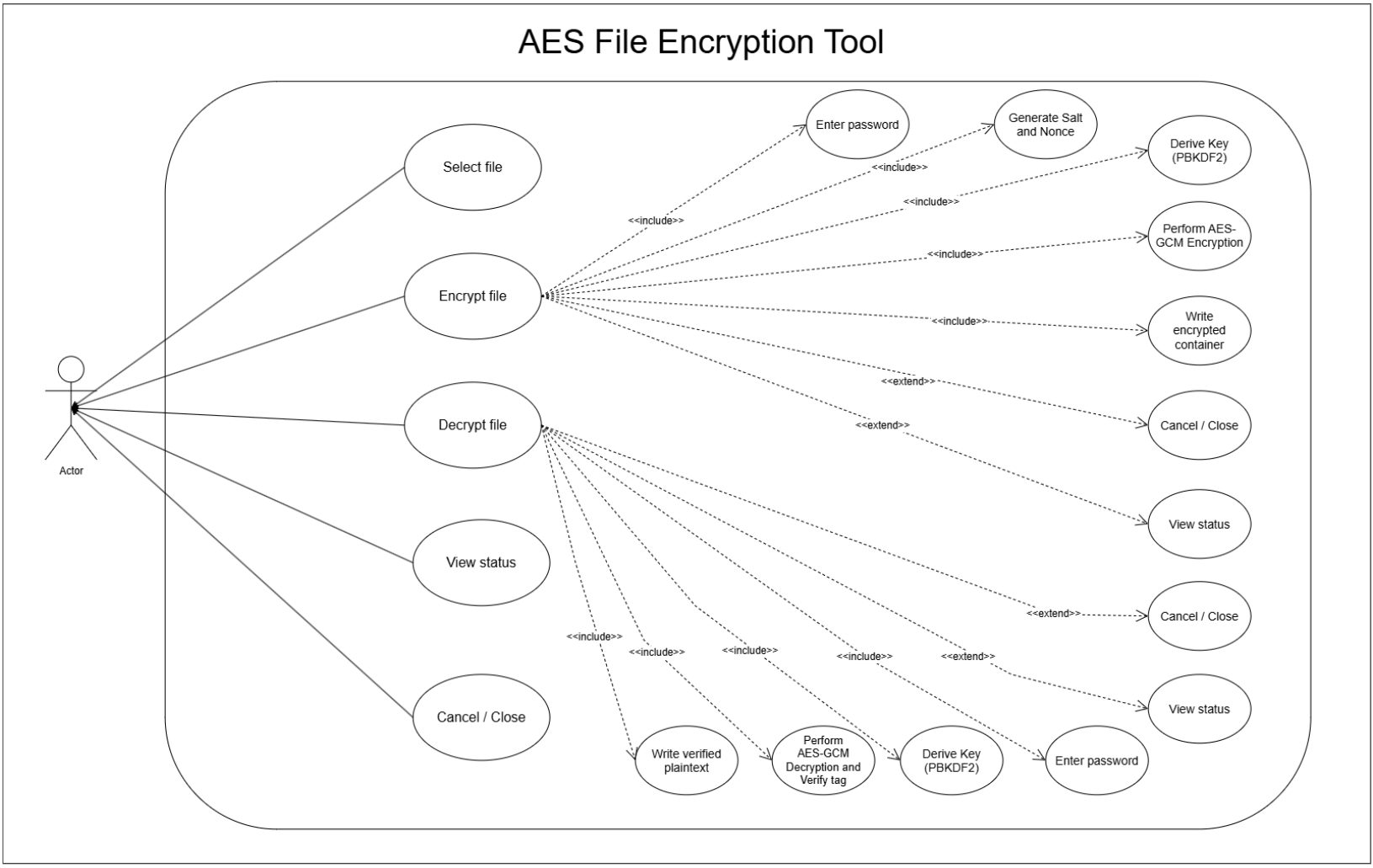

Figure 1. Use Case Diagram of the AES-Based Secure File Encryption and Decryption Tool.

5.2 Data Flow Diagram

The Data Flow Diagram represents the movement of files, passwords, derived keys, and encrypted data through the system. The main processes are file selection and password input, key derivation, AES-GCM encryption, encrypted file formatting, encrypted file parsing, key derivation during decryption.

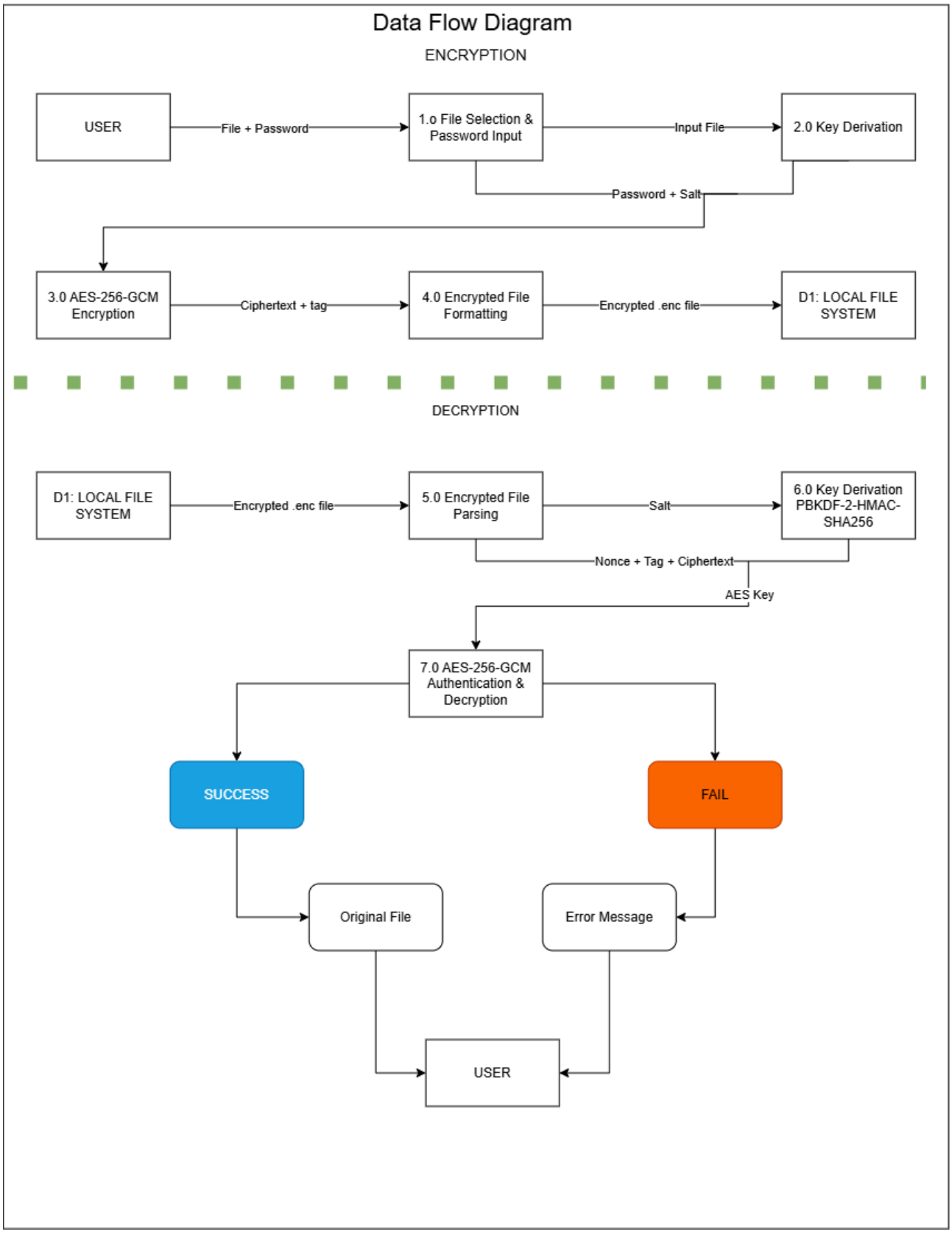

Figure 2. Data Flow Diagram of the AES-Based Secure File Encryption and Decryption Tool.

5.3 Application Flow Diagram

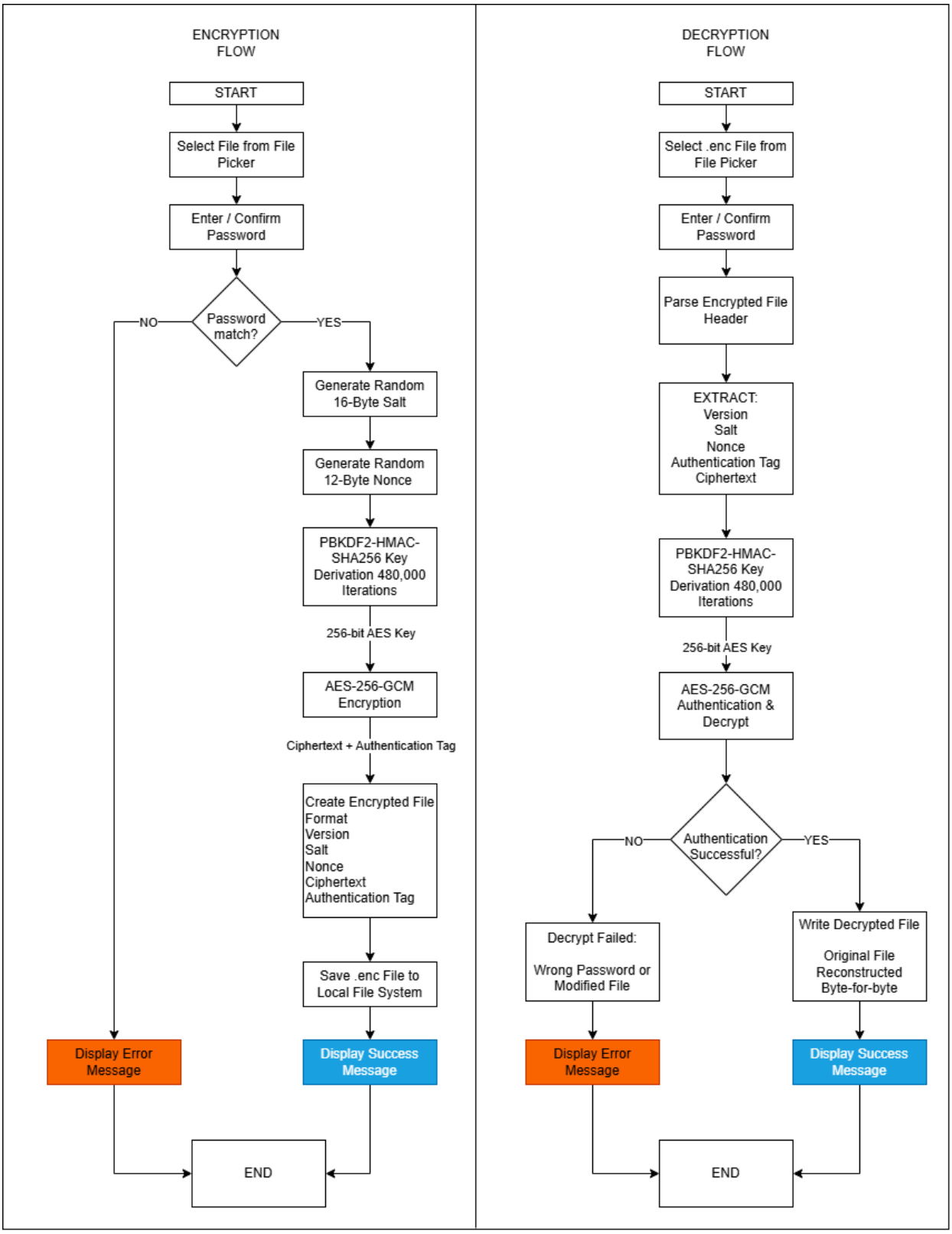

Figure 3. Application Flow Diagram for Encryption and Decryption.

6. Encryption Process Design

Step 1 – File and Password Input

The user selects a source file and enters a password twice. The application checks that a file exists and that both password entries match.

Step 2 – Salt Generation

A cryptographically secure random 16-byte salt is generated. The salt is stored with the encrypted file because it is required for future key derivation.

Step 3 – Key Derivation

PBKDF2-HMAC-SHA256 processes the password and salt using the selected iteration count to produce a 32-byte (256-bit) AES key.

Step 4 – Nonce Generation

A fresh 12-byte nonce is generated for the AES-GCM operation. The nonce is stored with the encrypted data and is not treated as secret.

Step 5 – AES-GCM Encryption

The plaintext file bytes are encrypted using AES-256-GCM. The operation produces ciphertext and a 16-byte authentication tag.

Step 6 – File Formatting

The application writes the version/metadata fields, salt, nonce, authentication tag, and ciphertext to the encrypted output file.

Step 7 – Completion

The encrypted file is saved separately, while the original file remains unchanged by default.

7. Decryption Process Design

Step 1 – Encrypted File Selection

The user selects a file with the expected encrypted format and enters the password.

Step 2 – File Parsing

The application validates the file header and extracts the salt, nonce, authentication tag, and ciphertext.

Step 3 – Key Derivation

The password and stored salt are passed through PBKDF2-HMAC-SHA256 using the stored derivation parameters.

Step 4 – Authentication and Decryption

AES-GCM verifies the authentication tag while decrypting the ciphertext. The result is accepted only if authentication succeeds.

Step 5 – Success or Failure

If authentication succeeds, the plaintext is written to the chosen destination. If authentication fails, an error is shown and no final decrypted file is created.

8. Cryptographic Design

8.1 Password and Key Derivation

The user's password is not used directly as the AES key. PBKDF2-HMAC-SHA256 is used to transform the password into a 32-byte key. A new random salt is generated for each encryption operation. The design uses the iteration count specified in the project proposal, with the implementation keeping the value in the encrypted file metadata so that the same derivation process can be reproduced.

The password itself must never be stored in the encrypted file. Only the non-secret salt and parameters required to reproduce the derivation are stored.

8.2 AES-256-GCM

AES-256-GCM provides both confidentiality and authentication. AES uses a 256-bit key, while GCM produces an authentication tag that allows the application to detect modification of encrypted data. The Python cryptography library will be used instead of a custom AES implementation.

8.3 Salt and Nonce Generation

The salt and nonce are generated using a cryptographically secure random-number source. The proposed design uses a 16-byte salt and a 12-byte nonce. The salt may be reused only as stored metadata for its corresponding encrypted file; a fresh salt is generated for every new encryption operation. A fresh nonce is generated for every AES-GCM encryption operation.

8.4 Authentication Tag

The authentication tag is a security-critical value. It is stored with the encrypted data and must be supplied unchanged during decryption. If the tag verification fails, the application treats the operation as unsuccessful. This covers incorrect passwords and modifications to the ciphertext or other authenticated encryption inputs.

9. Encrypted File Format

The encrypted file uses a self-contained binary structure so that no separate key file is required. A version field is recommended so that future versions of the application can identify changes to the format.

Field	Example Size	Purpose	Secret?
Magic / Version	Variable	Identifies the file type and format version.	No
PBKDF2 Parameters	Variable	Stores parameters required to reproduce key derivation.	No
Salt	16 bytes	Used with the password to derive the AES key.	No
Nonce	12 bytes	Required for the AES-GCM operation.	No
Authentication Tag	16 bytes	Used to verify ciphertext authenticity.	No
Ciphertext	Variable	Encrypted contents of the original file.	Protected

The exact byte layout and length encoding should be documented in the implementation so that parsing is deterministic and malformed files can be rejected safely.

10. User Interface Design

Interface Component	Design
Main Window	Provides clear choices for Encrypt and Decrypt operations.
File Selection	Uses a file-picker dialog and displays the selected filename/path.
Password Entry	Uses masked input. Encryption includes password confirmation.
Progress/Status Area	Shows the current operation without exposing cryptographic secrets.
Result Messages	Reports successful completion or a general failure reason.
Output Selection	Allows the user to choose where the resulting encrypted or decrypted file is saved.

11. Error Handling and Security Controls

Condition / Risk	Control
Wrong Password	AES-GCM authentication failure is returned as a general decryption failure. No plaintext output is created.
Modified Ciphertext	Authentication fails and the encrypted file is rejected.
Malformed File	Header and length validation are performed before cryptographic processing.
Missing Source File	The application reports that the selected file cannot be accessed.
Permission Error	The application displays a user-friendly message instead of terminating unexpectedly.

Interrupted Write	Temporary output is used and the final file is created/renamed only after successful completion.
Password Exposure	Password values are masked and are not stored in the encrypted file or ordinary application logs.
Accidental Overwrite	The application should ask for confirmation before overwriting an existing destination.

12. Testing and Evaluation Design

Testing will evaluate correctness, security, usability, performance, and robustness. The tests will compare the original and decrypted files using SHA-256 hashes to confirm byte-for-byte recovery. Security tests will intentionally use incorrect passwords and modify encrypted data to confirm that authentication failures are handled correctly.

Test	Method	Expected Result
Round-trip test	Encrypt then decrypt a file and compare SHA-256 hashes.	Original and decrypted hashes match.
Wrong password	Attempt decryption with an incorrect password.	Authentication fails and no final plaintext file is produced.
Tampered ciphertext	Modify one or more ciphertext bytes.	Authentication fails.
Tampered tag	Modify the authentication tag.	Authentication fails.
Different file types	Test TXT, PDF, image, and archive files.	All files decrypt correctly.
File sizes	Test small and large files, including performance benchmarks.	Operations complete without corruption.
Missing file	Remove or move the selected input before operation.	Clear error and no crash.
Interrupted operation	Simulate an interrupted write where practical.	No misleading incomplete final output.

13. Design Limitations

The system is designed as an academic desktop prototype rather than an enterprise key-management platform. It does not provide password recovery, cloud integration, multi-user access control, hardware security modules, or protection against attackers who already control an unlocked running system. The security of password-based encryption also depends heavily on the quality of the user's password. These limitations are intentional and are consistent with the scope established in the project proposal.

14. Conclusion

The system design converts the cryptographic requirements of the proposed project into a practical desktop architecture. The design combines a simple Tkinter interface with PBKDF2-HMAC-SHA256 key derivation and AES-256-GCM authenticated encryption. Random salts and fresh nonces are generated for each

encryption operation, while the encrypted file contains the non-secret information needed for future decryption. Authentication is treated as a mandatory condition before decrypted output is released.

The three system diagrams—Use Case Diagram, Data Flow Diagram, and Application Flow Diagram—provide complementary views of the system and will be inserted into this report before submission. Together with the requirements, cryptographic design, file format, interface design, error controls, and test plan, they provide the foundation for the implementation phase.

15. References

Dworkin, M. (2007). *Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC* (NIST Special Publication 800-38D). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-38D

Kaliski, B. (2017). *PKCS #5: Password-based cryptography specification version 2.1* (RFC 8018). Internet Engineering Task Force. https://doi.org/10.17487/RFC8018

National Institute of Standards and Technology. (2001). *Advanced Encryption Standard (AES)* (FIPS PUB 197). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.197

National Institute of Standards and Technology. (2010). *Recommendation for password-based key derivation: Part 1, storage applications* (NIST Special Publication 800-132). U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-132

OWASP Foundation. (n.d.). *Cryptographic storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html

OWASP Foundation. (n.d.). *Password storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Python Cryptographic Authority. (n.d.). *Authenticated encryption with associated data (AEAD)*. cryptography. https://cryptography.io/en/latest/hazmat/primitives/aead/